

Invoice Canonicalization Agent

Deterministic first. AI only for uncertainty.
Human approvals become cheaper future knowledge.

TIER 1

Approved lookup

no LLM

TIER 2

Retrieval / AI

bounded

TIER 3

Human learning

governed

Production-ready concept + tested reference implementation

Every product is checked before canonicalization

Known or wild, the same extraction and arithmetic quality gate runs first.

THE BUSINESS MESS



Invoices arrive with noisy descriptions and full financial context.

If extraction is wrong, every downstream match starts from bad evidence.

UNIVERSAL QUALITY GATE



Every line — tamed or wild — is checked first

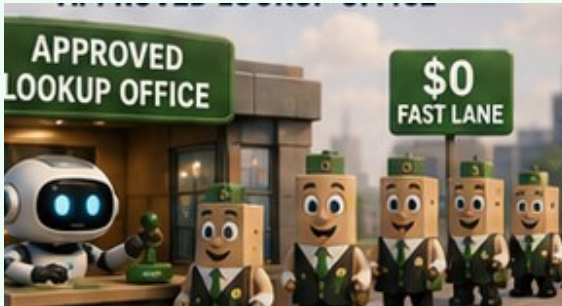
- $\text{quantity} \times \text{unit price} = \text{line total}$
- $\Sigma \text{ line totals} = \text{subtotal}$
- discount / tax / shipping reconciled separately

Only clean product-line evidence enters canonicalization.

Known = fast lane. Unknown = uncertainty lab.

After the same quality gate, approved aliases are deterministic; only unknowns see retrieval or AI.

KNOWN / TAMED



Approved Lookup

Exact tenant-scoped alias →
stable canonical product ID.

0 LLM CALLS · FAST

UNKNOWN / WILD



Fast lane blocked

Unknown is routed
deliberately — not treated as
a failed invoice.

UNCERTAINTY ROUTE

TIER 2 · UNCERTAINTY LAB



1 Approved
knowledge



2 Retrieve evidence



3 Bounded AI /
abstain

Investigate once → promote approved
knowledge → next time use Tier 1.

Approval turns a wild product into reusable knowledge

First cycle: evidence + review. Second cycle: exact alias, no model, no review.

FIRST CYCLE



Human approves the mapping

Black Leather Jacket Midnight

↓ **Black Leather Jacket**

Approval is promoted to the tenant-scoped alias registry.



SECOND CYCLE



Next time: exact alias

0

LLM calls

0

human reviews

\$0

model cost

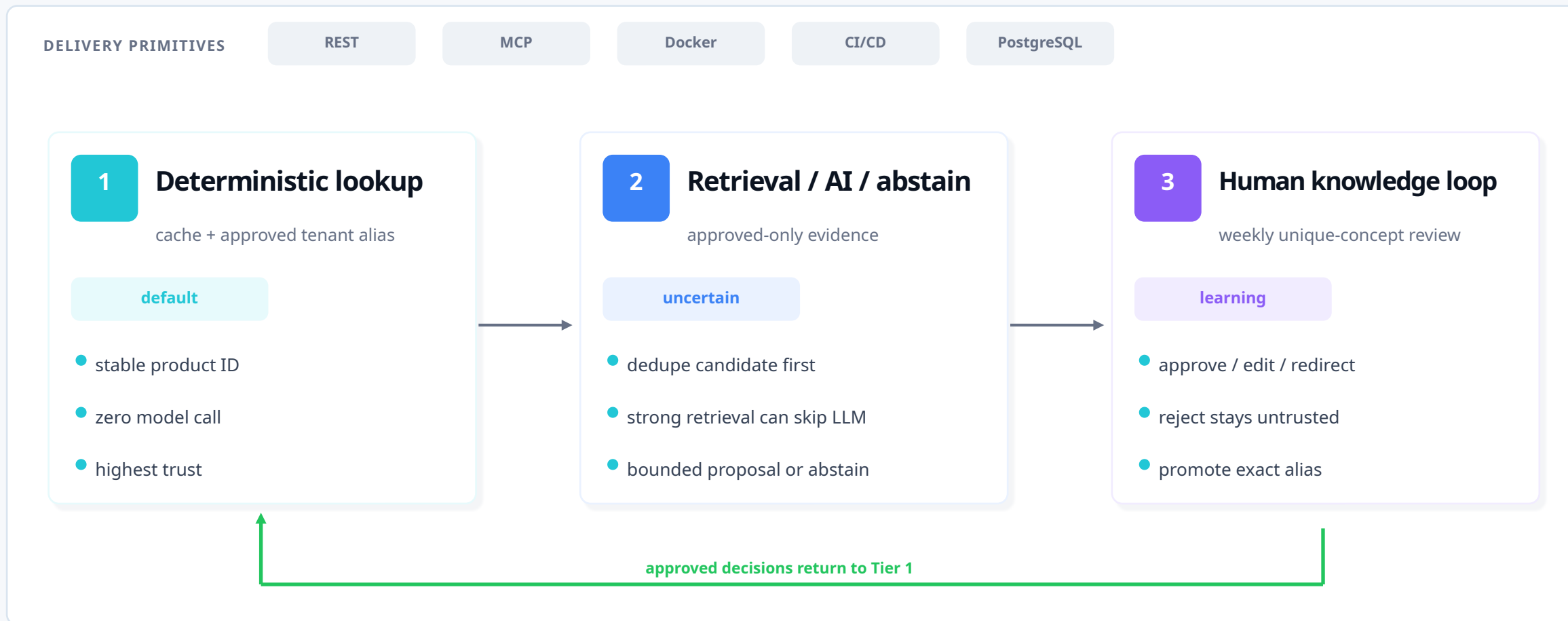


cost curve

Model spend follows unique unresolved concepts — not repeated invoice volume.

Three-tier canonicalization architecture

That story maps directly to three production tiers; delivery primitives stay outside the business logic.



Story -> code: Approved Lookup = approved aliases | Uncertainty Lab = approved retrieval + bounded AI / abstention | Human Approval = review queue -> knowledge promotion.

Challenge file: integration smoke test, not ML accuracy

The expected mappings are seeded approved aliases - this verifies deterministic replay and extraction integrity.

INVOICE

01. Juli 2025

19283746552

Due in 15 days

LOGO

Testinger GmbH
123 Test Street
Testingen, Testern,
0815
+00 1234 5678 00
testinger.ts
email@testinger.ts

BILL TO

Mr Obertestinger
Recipient Corp.
123 Empfängergerasse
0815 Recipstan
+00 1234 8876 0912,
recip@recipcoopr.co

SHIP TO

Mr Obertestinger
Recipient Corp.
123 Empfängergerasse
0815 Austria
+00 1234 8876 0912,
recip@recipcoopr.co

DESCRIPTION	QTY	UNIT PRICE	TOTAL
Highlife Steel Accessories	1	10	10.00
Sneaker "Unstoppable"	2	11	22.00
T-Shirt White "Polarbear"	3	12	36.00
T-Shirt Beige "Grizzly"	4	13	52.00
Shorts "El Camino"	5	14	70.00
Socks, black	6	15	90.00
SUBTOTAL			280.00
DISCOUNT			13.00
SUBTOTAL LESS DISCOUNT			267.00
TAX RATE			20.00%
TOTAL TAX			53.40
SHIPPING/HANDLING			12.00
Balance Due			\$ 332.40

Company Signature

Client Signature

Deterministic replay

Highlife Steel Accessories	→ Highlife Components	NO LLM
Sneaker "Unstoppable"	→ Performance Sneakers	NO LLM
T-Shirt White "Polarbear"	→ White Tee	NO LLM
T-Shirt Beige "Grizzly"	→ Beige Tee	NO LLM
Shorts "El Camino"	→ Casual Shorts	NO LLM
Socks, black	→ Crew Socks	NO LLM

6 / 6

seeded aliases

smoke test

0

LLM calls

canonicalization

PASS

extraction gate

6/6 row equations

280=280

subtotal check

Decimal

Interpretation: proves integration + deterministic replay. It does not estimate unseen-model accuracy.

Measured reliability: regression gates, not vanity metrics

We distinguish deterministic smoke tests from model-quality claims.

107

tests passed

full offline gate

26→1

repeated unknowns

observations → proposal

11

routing cases

curated regression

100%

routing accuracy

on curated cases

0

unexpected LLM calls

target 0

What the results prove

- approved knowledge replays deterministically
- routing chooses the intended path in regression cases
- repeated unknowns do not trigger repeated model calls
- unsafe input can abstain before generation
- package builds and installs in isolation

What we do NOT claim

- 6/6 challenge = ML accuracy (it is a seeded smoke test)
- 11 cases predict production accuracy at scale
- vector RAG outperforms lexical retrieval (not benchmarked)
- live model cost/latency is validated by offline fixtures

Engineering gate: 83% branch-aware coverage (threshold $\geq 80\%$). Ruff, mypy, and Docker were unavailable locally; strict CI / make assess-full requires them.

Why this is stronger than the original brief

The implementation stays faithful to the simple requirement while proving a credible path to production.

ORIGINAL ASSIGNMENT

- same invoice should produce one reproducible description
- different clients and business partners
- conceptualize a solution; prototype optional
- fix randomness from direct LLM generation

OUR RESPONSE

- approved canonical IDs + aliases make replay deterministic
- tenant-specific taxonomy prevents client collisions
- AI is bounded to the unknown tail and deduplicated
- human decisions become trusted future exact matches
- operational guardrails + packaging demonstrate production readiness

THE TAKEAWAY

The goal is not to make the LLM more consistent. It is to need the LLM less often.

APPENDIX

Technical depth when the interviewer asks

Unknown lifecycle, review workflow, tenant isolation, delivery packaging, testing, retrieval, security, CI/CD, and limitations.

18-stage quality contract

test matrix

security + cost

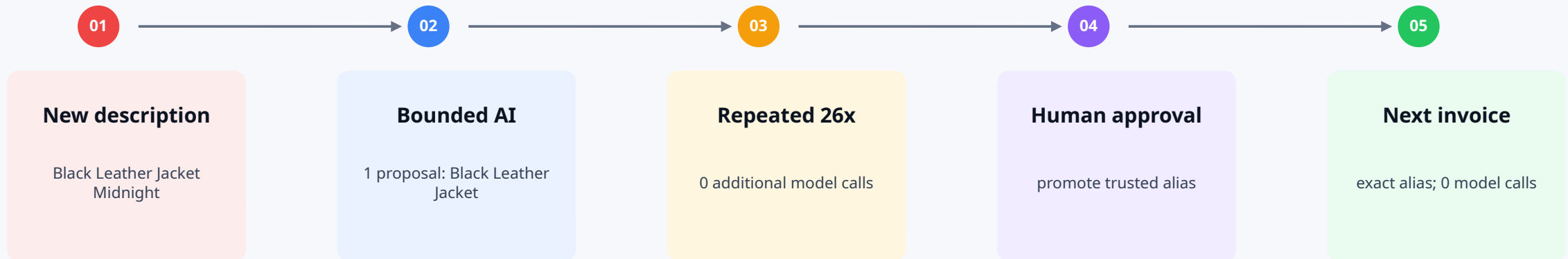
retrieval / RAG

review data model

limitations

An unseen description uses AI once - then becomes deterministic

The same pending concept is reused before approval; approval converts it into exact knowledge.



Scalability rule

The model is called once per unique pending concept - not once per invoice row.

This is the declining cost curve in operational form.

**26
observations
1 proposal**

**after approval
0 future calls**

Humans approve knowledge - not invoices

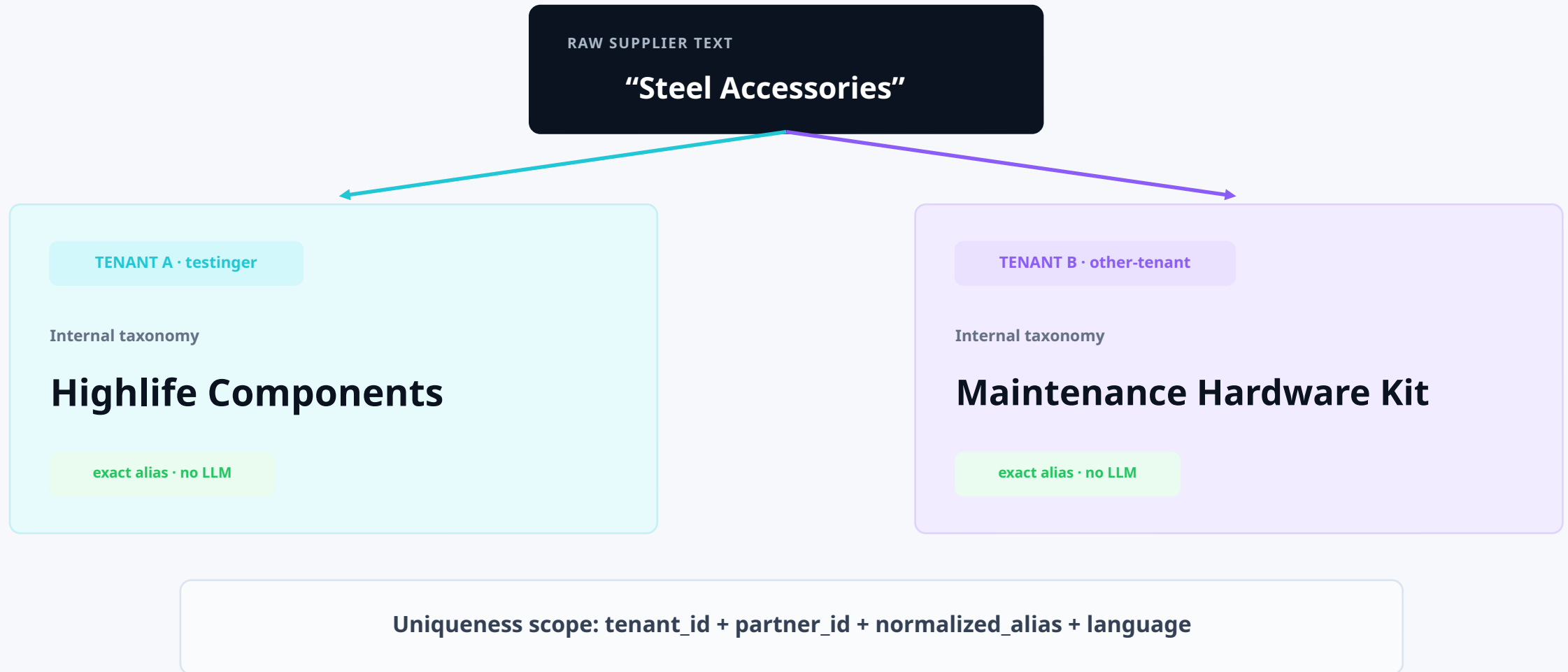
A weekly queue contains unique unresolved concepts with evidence, impact, and flexible actions.

PENDING KNOWLEDGE REVIEW						
				27 unique concepts	71,482 lines affected	
Priority	Observed variants	Proposal	Score	LLM	Occurrences	Status
HIGH	Air Runner Pro Black AirRunner Pro blk	Performance Sneakers	98.2%	Yes	18,400	approve
HIGH	Black Leather Jacket Midnight	Black Leather Jacket	73.0%	Yes	26	waiting
MED	Athletic crew socks	Crew Socks	97.8%	No	127	waiting
Reviewer actions						
Approve existing Redirect Create new Edit + approve Reject Defer						

Completed decisions leave the active queue but remain archived + audited.

Tenant taxonomies prevent cross-client collisions

The raw string is not a global key: alias lookup is scoped by tenant + partner + language.



Authentication binds the request to a tenant; PostgreSQL RLS is a second defense-in-depth layer.

Core solution vs. production packaging

This avoids over-engineering the story: the business logic stays small; the rest is delivery discipline.

CORE BUSINESS LOGIC

- canonical product IDs
- tenant-scoped approved aliases
- normalization + retrieval policy
- bounded AI for unknowns
- human knowledge promotion

OPERATIONAL GUARDRAILS

- extraction arithmetic gate
- auth + tenant binding
- budgets / timeouts / retries
- validation + abstention
- audit + monitoring

DELIVERY PRIMITIVES

- REST API
- MCP integration boundary
- Docker / Compose
- GitHub CI/CD
- PostgreSQL migrations

Interview rule: lead with the first column. Use the other two only when asked how you would productionize it.

Internal quality sequence - 18 machine-checked stages

Appendix detail: this is an implementation contract, not the primary business architecture.

01-06

Input + evidence

- auth + tenant binding
- file validation
- deterministic parsing
- Decimal line gate
- model extraction fallback
- persist invoice envelope

07-10

Deterministic state

- normalize text
- version-safe cache
- approved exact alias
- atomic pending claim

11-14

Uncertainty handling

- approved-only retrieval
- policy scoring
- bounded generation
- output/security validation

15-18

Governance

- staged review queue
- human action
- knowledge promotion
- audit + metrics

Testing strategy

Offline determinism for CI; live-provider evaluation remains opt-in.

Layer	What it proves	Property
Unit / contracts	normalization, money, budgets, prompt schemas	fast / deterministic
Integration	repository, canonicalization, review promotion	real application wiring
Document metamorphic	PDF / XLSX / DOCX / JSON / CSV / TXT	same business result across formats
AI fixtures	stored prompt examples + model outputs	no API-spend CI, stable regressions
Adversarial	prompt injection, PII/financial leakage, cross-tenant, malformed output	safe failure / abstention
Concurrency	same unknown arriving simultaneously	one candidate / one proposal
Packaging	wheel build + isolated install smoke	delivery integrity

Current verified gate: 107 tests passed · 83% branch-aware coverage · 11 curated routing cases all correct.

Security, reliability, and cost guardrails

These protect the product logic; they do not decide the canonical product.

SECURITY

- tenant-bound auth
- PostgreSQL RLS
- party PII kept out of prompts
- prompt-injection guard
- CSV formula neutralization

RELIABILITY

- Decimal financial fields
- line arithmetic gate
- commercial reconciliation
- schema + attribute validation
- immutable audit history

COST / LATENCY

- exact alias before retrieval
- dedupe before model
- max calls + dollar budget
- timeouts + bounded retries
- cache by taxonomy version

Retrieval / RAG boundary

Use the simplest retrieval method that meets false-match, latency, and recall targets.

CURRENT ASSESSMENT

- approved aliases only
- sequence similarity
- token overlap
- character trigrams
- category + protected attributes
- margin vs. second candidate

SCALE-UP OPTION

- SemanticScoreProvider interface exists
- PostgreSQL schema is pgvector-ready
- benchmark Recall@K + false match first
- measure latency + memory before enabling
- do not index unapproved model suggestions

Anti-feedback-loop rule: staging data never becomes retrieval evidence until a human promotes it.

Review data model and state transitions

Local CSV is an editable projection; production persistence is relational.

Candidate record

- normalized candidate key
- source variants + occurrence count
- affected value by currency
- retrieval / policy scores
- model + prompt provenance
- risk flags + evidence
- human overrides + notes

Human-editable final status

waiting

approve existing

redirect

create new

edit + approve

reject

defer

Processed rows leave the active work queue but remain in archive + audit history.

Concurrency: one active candidate per tenant + partner + normalized description.

Limitations and next validation

What still needs evidence before a real production rollout.

Area	Current evidence	Next validation
Production accuracy	11 curated cases are regression evidence, not population statistics	Build a blind labelled dataset across clients/categories/languages
Threshold calibration	current thresholds are configuration, not statistically calibrated probabilities	Optimize to a business target such as false auto-resolution < 0.1%
Live provider	offline fixtures prove contracts but not real latency/cost/drift	Run controlled live evaluation with approved model + pricing
Semantic retrieval	pgvector seam exists but is not benchmarked as better	Compare Recall@K / latency / false-match against lexical hybrid
Release environment	Ruff, mypy, Docker skipped in sandbox	Run make assess-full in CI / local Docker environment
Operational UX	CSV proves workflow but is not a polished reviewer product	Replace with dashboard when reviewer volume justifies it

Delivery and demo commands

Everything needed to inspect, test, demonstrate, and package the solution.

<code>./run_assessment.sh</code>	Offline quality gate: tests, coverage, evaluation, package smoke
<code>make interview-demo</code>	Three-tier lifecycle demonstration + machine-readable evidence
<code>make review-demo</code>	Generate the editable pending-knowledge CSV
<code>make review-process</code>	Apply reviewer decisions, archive completed items
<code>make assess-full</code>	Strict release gate including Ruff, mypy, Docker

Assessor-facing artifacts

- README.md
- docs/assessor_summary.md
- docs/invoice_data_model.md
- reports/final_delivery_check.md
- docs/verified_results.md
- docs/production_validation_plan.md
- presentation/PPTX + PDF

Version 1.0.0